Java Backend Architecture

Interview Series

Chapter 19.2

Infrastructure as Code

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 19.2 – Infrastructure as Code

Overview

Infrastructure as Code (IaC) enables provisioning and managing cloud infrastructure through declarative configuration files, ensuring reproducibility, version control, and automation. Terraform is the dominant tool for multi-cloud provisioning; Ansible for configuration management; Pulumi for code-first IaC in Java/Python. Understanding state management, modules, workspaces, and idempotency is expected at senior level.

Key Definitions

Term	Definition
Terraform	Declarative IaC tool. Defines desired state in HCL; plan/apply converges actual to desired.
Provider	Terraform plugin interfacing with cloud APIs (aws, google, azurerm, kubernetes).
Resource	A manageable infrastructure component (aws_instance, google_container_cluster).
Data source	Read-only reference to existing infrastructure not managed by this Terraform state.
Module	Reusable Terraform configuration. Encapsulates resources with input variables and outputs.
State	Terraform's record of managed infrastructure (terraform.tfstate). Source of truth for drift detection.
workspace	Isolated Terraform state per environment (dev/staging/prod) using same config.
Ansible	Agentless configuration management via SSH. Playbooks define tasks on inventory of hosts.
Idempotency	Running IaC multiple times produces the same result. Critical property for safe automation.
Pulumi	IaC using general-purpose languages (Python, TypeScript, Java). Programs describe infrastructure.

Diagram 1: Terraform Workflow

```
Developer
     |
     | 1. terraform init      (download providers, init backend)
     v
[Provider plugins downloaded]
     |
     | 2. terraform plan      (diff: desired vs current state)
     v
[Plan output: +create, ~update, -destroy]
     |
     | 3. Review plan
     v
[Approved?]
     |
     | 4. terraform apply     (execute plan, update state)
     v
[Infrastructure created/updated]
     |
     | State stored in S3 + DynamoDB lock (remote backend)
```

Diagram 2: Terraform Module Hierarchy

```
root module (main.tf)
  |
  +-- module "vpc"          (modules/vpc/main.tf)
  |     inputs: cidr, name
  |     outputs: vpc_id, subnet_ids
  |
  +-- module "eks"          (modules/eks/main.tf)
  |     inputs: vpc_id, subnet_ids, cluster_version
  |     outputs: cluster_endpoint, kubeconfig
  |
  +-- module "rds"          (modules/rds/main.tf)
        inputs: vpc_id, subnet_ids, db_password
        outputs: db_endpoint
```

Code Examples

Terraform -- EKS Cluster for Java Microservices:

```
# main.tf
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "prod/eks/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-locks"   # state locking
    encrypt        = true
  }
}

module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "java-prod-cluster"
  cluster_version = "1.29"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  eks_managed_node_groups = {
    java_workers = {
      instance_types = ["m5.xlarge"]
      min_size       = 3
      max_size       = 10
      desired_size   = 3
    }
  }
}
```

Ansible Playbook -- Spring Boot Deployment:

```
# deploy_java_app.yml
---
- hosts: java_servers
  become: true
  vars:
    app_version: "{{ lookup('env', 'APP_VERSION') }}"
    app_port: 8080

  tasks:
    - name: Deploy Spring Boot JAR
      copy:
        src: "target/app-{{ app_version }}.jar"
        dest: /opt/java-app/app.jar
        owner: javaapp
        mode: '0755'

    - name: Restart application service
      systemd:
        name: java-app
        state: restarted
        enabled: true

    - name: Wait for app health check
      uri:
        url: "http://localhost:{{ app_port }}/actuator/health"
        status_code: 200
      retries: 30
      delay: 5
```

Terraform Workspace per Environment:

```
# Create workspace
terraform workspace new staging
terraform workspace new prod

# Select workspace
terraform workspace select prod

# Use workspace in config
locals {
  env_config = {
    staging = { instance_type = "t3.medium", min_size = 1 }
    prod    = { instance_type = "m5.xlarge", min_size = 3 }
  }
  config = local.env_config[terraform.workspace]
}

resource "aws_autoscaling_group" "java_asg" {
  instance_type = local.config.instance_type
  min_size      = local.config.min_size
}
```

Interview Q&A – Infrastructure as Code

Q1. What is Terraform state and why must it be stored remotely?

Terraform state (terraform.tfstate) maps your HCL resources to actual infrastructure IDs. Terraform uses it to compute diffs (plan) and detect drift. Local state: works for solo dev but dangerous -- two engineers running apply simultaneously corrupt state. Remote backend (S3 + DynamoDB): encrypted at rest, state locking via DynamoDB prevents concurrent applies, versioned for recovery. Always use remote state for team/production Terraform.

Q2. Explain terraform plan vs terraform apply.

terraform plan: reads current state, queries cloud APIs for actual infrastructure, computes diff (+ create, ~ update, - destroy). Does NOT make changes. terraform apply: executes the plan, creates/updates/destroys resources, updates state file. Best practice: always run plan first and review before apply, especially for -destroy operations. In CI/CD: plan on PR (comment output), apply only on merge to main after approval.

Q3. What is the difference between resource and data source in Terraform?

resource manages a piece of infrastructure -- Terraform creates, updates, and destroys it. data source reads existing infrastructure not managed by this config -- read-only. Example: data aws_vpc { filter name=prod } references an existing VPC created by another team's Terraform. Use data sources to bridge between Terraform stacks. Never use data source to reference resources in the same config -- use direct references instead.

Q4. How do Terraform modules promote reusability?

Modules encapsulate related resources with input variables and outputs. The root module calls child modules: module eks { source = './modules/eks'; cluster_version = '1.29' } Public modules: Terraform Registry (terraform-aws-modules/eks). Version-pin modules: version = '~> 20.0'. Output values enable inter-module references (module.eks.cluster_endpoint). Modules eliminate copy-paste infrastructure across Java microservice environments.

Q5. What is Ansible idempotency and why is it important?

Idempotency: running a playbook N times has same result as running it once. Ansible modules check current state before acting: copy module only copies if checksum differs, systemd only restarts if service state changed. Non-idempotent: shell: apt-get install nginx runs unconditionally -- use apt: name=nginx state=present. Idempotency enables safe repeated playbook runs in CI/CD without fear of unintended side effects.

Q6. How do Ansible roles structure configuration management?

Roles provide a standard directory structure: tasks/main.yml, handlers/main.yml, templates/ (Jinja2), files/, vars/main.yml, defaults/main.yml, meta/main.yml. Install from Ansible Galaxy: ansible-galaxy install geerlingguy.java. Roles are reusable units: a java-app role handles JDK installation, service configuration, log rotation across all Java server deployments. Include in playbook: roles: [java-app].

Q7. What is the terraform taint command and when would you use it?

terraform taint resource_address marks a resource for recreation on next apply. Use when a resource is in an inconsistent state that Terraform's drift detection misses (EC2 instance with corrupted data, EKS node with broken kubelet). In Terraform 0.15+, replaced by terraform apply -replace='aws_instance.java_server'. Forces destroy + create even if configuration hasn't changed.

Q8. How does Pulumi differ from Terraform for Java teams?

Pulumi uses general-purpose languages (Java, Python, TypeScript) instead of HCL. Benefits: real control flow (loops, conditionals, functions), type safety, IDE autocompletion, unit testing of infrastructure code. State: managed by Pulumi Cloud or self-hosted (S3). Java example: new aws.eks.Cluster(name, args). Terraform's HCL has a shallower learning curve; Pulumi is better when IaC logic requires complex conditional logic that HCL handles awkwardly.

Q9. How do you prevent accidental terraform destroy in production?

lifecycle { prevent_destroy = true } on critical resources (aws_rds_instance, S3 bucket). Terraform rejects any plan that would destroy the resource. Remote backend with versioned state enables recovery. Separate state files per environment: production state in a different S3 path with restricted IAM policy. Require MFA for production terraform apply. Require PR approval before merging Terraform changes to main.

Q10. What is Ansible Vault and how is it used?

Ansible Vault encrypts sensitive files/variables (DB passwords, API keys) using AES-256. Encrypt file: ansible-vault encrypt vars/secrets.yml. Decrypt at runtime: ansible-playbook --ask-vault-pass or --vault-password-file. Encrypt individual variable: ansible-vault encrypt_string 'db_password'. Integration with CI/CD: store vault password in CI secret, pass via --vault-password-file

/dev/stdin. Vault password should be rotated and stored in HashiCorp Vault or AWS Secrets Manager.